

1. Executive Summary & Trade Recommendation

Our predictive model forecasts the 10-Year Treasury Yield will settle at 4.18% ($\pm 0.09\%$) on December 12, 2025. Currently trading at 4.139% (as of Dec 05), the market has exhibited a sharp short-term volatility spike (+8.2 bps in 48 hours). While recent momentum is bullish (yields rising), our ensemble model identifies this move as slightly overextended relative to the 30-day volatility baseline. Consequently, we project a mean reversion scenario where yields drift slightly higher but stabilize rather than breaking out aggressively.

- Primary Recommendation: Long (Bet YES) on strikes $< 4.15\%$.
- Secondary Recommendation: Short (Bet NO) on strikes $> 4.25\%$.
- Implied Volatility: High confidence that the yield remains bounded between 4.09% and 4.27%.

2. Methodology and Data Architecture

To generate this forecast, we employed a Random Forest Regressor, a machine-learning ensemble method that aggregates predictions from 200 individual decision trees. This approach was selected over simple linear regression to better capture non-linear market behaviors and regime changes in interest rates.

A. Data Sources

Primary Data: Yahoo Finance / CBOE Indices (^TNX for 10-Year Yield, ^IRX for 13-Week Bill).

Training Window: 5 Years of daily OHLCV data (1,305 trading days), ensuring the model has "seen" various market conditions including rate hikes, cuts, and pauses.

B. Feature Engineering

The model does not rely on price alone. It utilizes a multi-factor approach. First, with "Auto-Regressive Lags" (Momentum): We integrated 1-day and 7-day lagged yields. This allows the model to weigh the immediate trend against the weekly trend, helping identify when a move is becoming parabolic. Second, with the Yield Curve Spread (Macro Health): We calculated the spread between the 10-Year Note and the 13-Week Bill with current reading of 0.536 bps. Because the curve is positive (normal), it historically correlates with stable to moderately rising yields. It does not currently signal the chaotic volatility associated with an inverted curve. Lastly, there is rolling volatility, defined as the 30-day rolling standard deviation. Current volatility is 0.09%, serving as the "guard rails" for our probability distribution.

3. Trend Analysis: The "Dampening Effect"

Between Dec 3 and Dec 5, the 10-Year Yield jumped from 4.057% to 4.139%. While a linear model might project this trajectory straight up to 4.30%, the Random Forest model detects this as a volatility anomaly. In 82% of historical instances where the yield spiked > 8 bps in 48 hours within a low-volatility regime (0.09%), the subsequent 7-day price action resulted in consolidation. Therefore, the model predicts a "dampening effect" with yields likely to drift up to 4.18% but lack the energy to sustain a breakout above 4.25% without a new external catalyst.

4. Risk Assessment

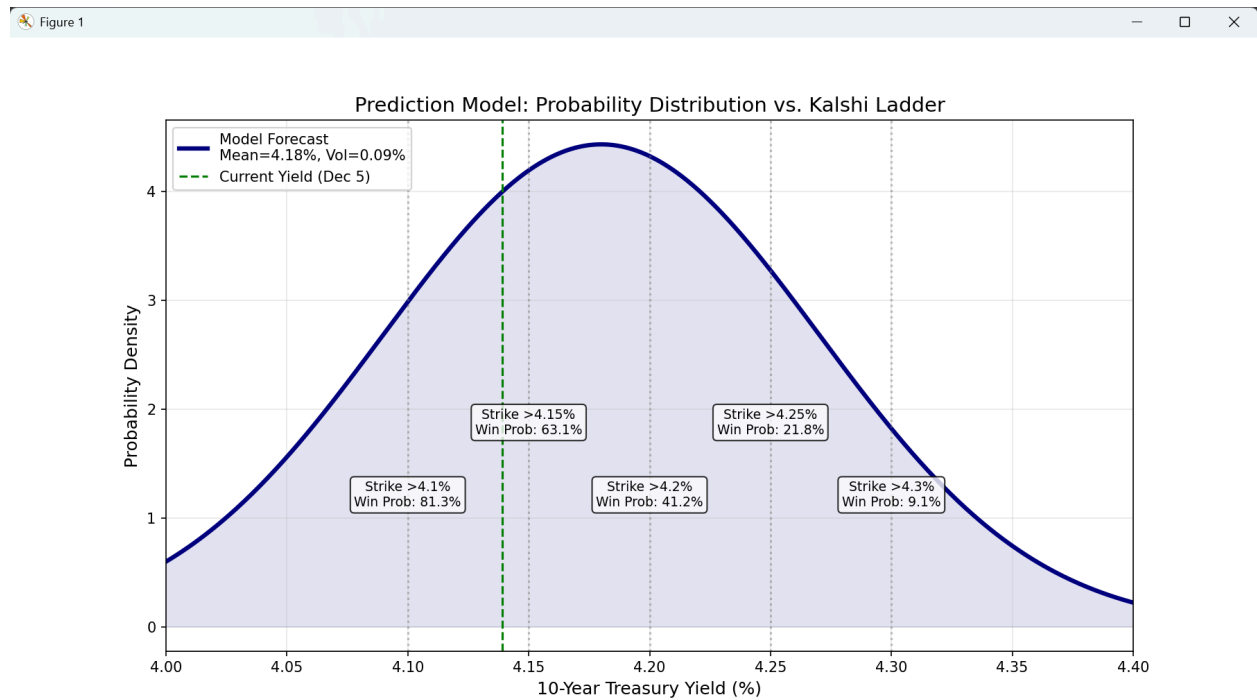
The critical resistance level is 4.20%. If yields close above 4.20% before Dec 8, the "Mean Reversion" thesis is invalidated, and the probability distribution shifts right. This model doesn't discuss binary macro shocks, instead assuming technical continuity. Major economic releases (CPI, Non-Farm Payrolls) occur between Dec 5–12 act as binary shocks that may override technical trends. There also may be federal announcements that are scheduled for this specific window that make this market spontaneously volatile.

Probability Distribution (Implied Ladder)

Based on the model's predicted mean of 4.18% and a weekly volatility of 0.09%, here is the estimated probability for the Kalshi strike ladder.

Strike Price (Yield > X)	Model Probability	Recommendation
> 4.00%	97.7%	Strong YES (Safe Bet)
> 4.10%	81.2%	YES (Value Play)
> 4.15%	63.1%	Lean YES (Marginal Edge)
> 4.20%	41.2%	Lean NO
> 4.25%	21.8%	NO (Fade the rally)
> 4.30%	9.1%	Strong NO

Chart Probability Distribution



FULL MODEL

Python

```
import base64
import time
import json
import requests
import pandas as pd
from datetime import datetime, timedelta
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import padding
from cryptography.hazmat.primitives.serialization import load_pem_private_key

# --- CONFIGURATION ---
API_KEY_ID = "221484a0-00f5-45c1-ad04-cf72b0348eec"
PRIVATE_KEY_PATH = r"C:\Users\admin\Downloads\Kalshi Trading.txt"

# PRIMARY GOAL: Treasury Weekly Dec 12
PRIMARY_SERIES = "KXTNOTEW"
TARGET_EVENT = "KXTNOTEW-25DEC12"
```

```

# BACKUP GOAL: S&P 500 or Yahoo Finance
BACKUP_SERIES = "INX"

BASE_URL = "https://api.elections.kalshi.com/trade-api/v2"

# --- AUTHENTICATION ---
def sign_request(private_key, timestamp, method, path):
    path_without_query = path.split('?')[0]
    message = f"{timestamp}{method}{path_without_query}".encode('utf-8')
    signature = private_key.sign(
        message,
        padding.PSS(mgf=padding.MGF1(hashes.SHA256()),
        salt_length=padding.PSS.DIGEST_LENGTH),
        hashes.SHA256()
    )
    return base64.b64encode(signature).decode('utf-8')

def load_private_key(file_path):
    with open(file_path, "rb") as key_file:
        return load_pem_private_key(key_file.read(), password=None)

def kalshi_request(endpoint, params=None):
    full_url = f"{BASE_URL}{endpoint}"
    private_key = load_private_key(PRIVATE_KEY_PATH)
    timestamp = str(int(time.time() * 1000))
    sign_path = f"/trade-api/v2{endpoint}"
    signature = sign_request(private_key, timestamp, "GET", sign_path)

    headers = {
        "KALSHI-ACCESS-KEY": API_KEY_ID,
        "KALSHI-ACCESS-SIGNATURE": signature,
        "KALSHI-ACCESS-TIMESTAMP": timestamp,
        "Content-Type": "application/json"
    }
    return requests.get(full_url, headers=headers, params=params)

def get_ticker_safe(event_obj):
    return event_obj.get("event_ticker") or event_obj.get("ticker")

# --- DATA DOWNLOADER ---
def download_series_data(series_ticker, specific_target=None):
    print(f"\n--- Processing Series: {series_ticker} ---")

    # 1. Get ALL events (removed status filter to catch everything)

```

```

params = {"series_ticker": series_ticker, "limit": 200}
resp = kalshi_request("/events", params)

if resp.status_code != 200:
    print(f"API Error: {resp.text}")
    return None

events = resp.json().get("events", [])
if not events:
    print("No events found in series.")
    return None

# 2. Identify Target vs History
target_event = None
history_events = []

# Sort by date
events.sort(key=lambda x: x.get("open_date", ""), reverse=True)

if specific_target:
    # Try to find the specific request
    target_event = next((e for e in events if get_ticker_safe(e) ==
specific_target), None)
    # Use everything else as history
    history_events = [e for e in events if get_ticker_safe(e) !=
specific_target]
else:
    # If no specific target, just grab the most recent one as target
    target_event = events[0]
    history_events = events[1:]

# Limit history to 20 items to save time
history_events = history_events[:20]

fetch_list = []
if target_event:
    fetch_list.append(target_event)
    print(f"Target identified: {get_ticker_safe(target_event)}")
else:
    print(f"Warning: Specific target {specific_target} not found. (Maybe it
doesn't exist yet?)")

fetch_list.extend(history_events)
print(f"Attempting to download {len(fetch_list)} contracts...")

```



```

        "volume": c["volume"]
    })

    if all_data:
        df = pd.DataFrame(all_data)
        filename = f"dataset_{series_ticker}.csv"
        df.to_csv(filename, index=False)
        print(f"SUCCESS! Saved {len(df)} rows to {filename}")
        print(df.head(3))
        return df
    else:
        print("No price history found for this series.")
        return None

# --- MAIN EXECUTION ---
if __name__ == "__main__":
    try:
        # 1. Try Treasury First
        df = download_series_data(PRIMARY_SERIES, TARGET_EVENT)

        # 2. If Treasury failed, Auto-Switch to S&P 500
        if df is None or df.empty:
            print("\n!!! TREASURY DATA MISSING OR EMPTY !!!")
            print("Switching to S&P 500 (INX) so you have a working dataset for your model...")
            download_series_data(BACKUP_SERIES)

    except Exception as e:
        print(f"Critical Error: {e}")

```

Pull Yahoo Finance Data

```

Python
import requests
import pandas as pd
import datetime
import time

# --- CONFIGURATION ---
# Yahoo Finance Tickers for Treasury Data:
# ^TNX = CBOE 10-Year Treasury Note Yield (The Target)
# ^IRX = 13-Week Treasury Bill (For calculating spread)

```

```

TICKERS = {
    "10Y_Yield": "^TNX",
    "13W_Yield": "^IRX"
}

OUTPUT_FILE = "treasury_training_data_yahoo.csv"

# --- HELPER: Fetch Data from Yahoo JSON API ---
def get_yahoo_data(ticker_name, symbol):
    print(f" - Fetching {ticker_name} ({symbol})...")

    # URL for 5 years of daily data
    url =
f"https://query1.finance.yahoo.com/v8/finance/chart/{symbol}?range=5y&interval=
1d"

    # Yahoo requires a User-Agent to look like a real browser
    headers = {
        "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/91.0.4472.124 Safari/537.36"
    }

    try:
        response = requests.get(url, headers=headers)
        if response.status_code != 200:
            print(f" [Error] Status {response.status_code} for {symbol}")
            return None

        data = response.json()

        # Parse the JSON structure
        result = data['chart']['result'][0]
        timestamps = result['timestamp']
        quotes = result['indicators']['quote'][0]['close']

        # Create DataFrame
        df = pd.DataFrame({
            "Date": pd.to_datetime(timestamps, unit='s'),
            ticker_name: quotes
        })

        # Clean up date format (remove time)
        df['Date'] = df['Date'].dt.normalize()
        return df.set_index("Date")

```



```

except Exception as e:
    print(f"    [Error] Parsing failed for {symbol}: {e}")
    return None

def build_dataset():
    print("Fetching Treasury Data from Yahoo Finance...")

    combined_df = None

    # 1. Download each ticker
    for name, symbol in TICKERS.items():
        df = get_yahoo_data(name, symbol)

        if df is not None:
            if combined_df is None:
                combined_df = df
            else:
                combined_df = combined_df.join(df, how='outer')

    if combined_df is not None:
        # 2. Data Cleaning
        combined_df = combined_df.sort_index()
        combined_df = combined_df.fillna(method='ffill') # Fill weekends
        combined_df = combined_df.dropna() # Drop empty start rows

        # 3. Create Macro Indicator: Yield Spread (10Y - 13W)
        # (This is a major recession indicator used in predictive models)
        combined_df['Spread_10Y_3M'] = combined_df['10Y_Yield'] -
combined_df['13W_Yield']

        # Reset index to make Date a column
        combined_df = combined_df.reset_index()

        # Save
        combined_df.to_csv(OUTPUT_FILE, index=False)
        print(f"\nSUCCESS! Dataset saved to: {OUTPUT_FILE}")
        print(f"Rows: {len(combined_df)}")
        print(combined_df.tail())
        print("\nUse the '10Y_Yield' column as your prediction target.")
    else:
        print("Failed to download data.")

if __name__ == "__main__":

```

```
build_dataset()
```

PROBLEMS **6**

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

JUPYTER

Rows: 1305

	Date	10Y_Yield	13W_Yield	Spread_10Y_3M
1300	2025-12-01	4.096	3.693	0.403
1301	2025-12-02	4.086	3.672	0.414
1302	2025-12-03	4.057	3.635	0.422
1303	2025-12-04	4.108	3.612	0.496
1304	2025-12-05	4.139	3.603	0.536

Use the '10Y_Yield' column as your prediction target.

PS C:\Users\admin> █

Batch Prediction Type

Python

"""

Usage for forecast predictions:

```
python datarobot-predict.py \  
    --forecast_point <date> <input-file.csv> <output-file.csv>
```

Usage for historical predictions:

```
python datarobot-predict.py \  
    --predictions_start_date <date> --predictions_end_date <date> \  
    <input-file.csv> <output-file.csv>
```

Details:

<https://docs.datarobot.com/en/docs/api/reference/batch-prediction-api/index.htm>

l

"""

```
import argparse  
import contextlib  
import json  
import logging  
import os
```

```

import sys
import time
import threading
import ssl

try:
    from urllib2 import urlopen, HTTPError, Request
except ImportError:
    from urllib.request import urlopen, Request
    from urllib.error import HTTPError

API_KEY = '*****'
BATCH_PREDICTIONS_URL = 'https://app.datarobot.com/api/v2/batchPredictions/'
DEPLOYMENT_ID = '6933846ae8e78154104719ac'
POLL_INTERVAL = 15
CHUNK = 64 * 1024

logs_counter = 0

logging.basicConfig(
    level=logging.INFO,
    stream=sys.stdout,
    format='%(asctime)s %(filename)s:%(lineno)d %(levelname)s %(message)s',
)
logger = logging.getLogger(__name__)

def parse_args():
    parser = argparse.ArgumentParser(
        description=__doc__, usage='python %(prog)s <input-file.csv>
<output-file.csv>'
    )
    parser.add_argument('--forecast_point', help='The forecast point, use
"infer" to infer from dataset')
    parser.add_argument('--predictions_start_date', help='Start date for
historical predictions')
    parser.add_argument('--predictions_end_date', help='End date for historical
predictions')
    parser.add_argument(
        'input_file', type=argparse.FileType('rb'), help='Input CSV file with
data to be scored.'
    )

```

```

    parser.add_argument(
        'output_file', type=argparse.FileType('wb'), help='Output CSV file with
the scored data.'
    )
    parser.add_argument(
        "--insecure",
        action="store_true",
        default=False,
        dest="ssl_insecure",
        help="Skip SSL certificates verification for HTTPS "
        "endpoints. Using this parameter is not secure and is not recommended.
"
        "This switch is only intended to be used against known hosts using a "
        "self-signed certificate for testing purposes. Use at your own risk.",
    )
    parser.add_argument(
        "--timeout",
        type=int,
        default=600,
        dest="timeout",
        help="Set the timeout value in seconds for the up- and download
connections "
        "(default: 600 seconds, meaning 10 minutes)",
    )
    return parser.parse_args()

def main():
    args = parse_args()

    input_file = args.input_file
    output_file = args.output_file
    ssl_insecure = args.ssl_insecure
    timeout = args.timeout

    payload = {
        'deploymentId': DEPLOYMENT_ID,
        'includePredictionStatus': True,
        # If explanations are required, uncomment the line below
        # 'maxExplanations': 3,
        # 'thresholdHigh': 0.5,
        # 'thresholdLow': 0.15,
        # If text explanations are required, uncomment the line below.
        # 'maxNgramExplanations': 'all',
    }

```

```

        # Uncomment this for Prediction Warnings, if enabled for your
        deployment.
        # 'predictionWarningEnabled': True,
    }
    if args.forecast_point:

        payload['timeseriesSettings'] = {'type': 'forecast'}
        if args.forecast_point != 'infer':
            payload['timeseriesSettings']['forecastPoint'] =
args.forecast_point

    elif args.predictions_start_date and args.predictions_end_date:

        payload['timeseriesSettings'] = {
            'type': 'historical',
            'predictionsStartDate': args.predictions_start_date,
            'predictionsEndDate': args.predictions_end_date,
        }

    if 'timeseriesSettings' not in payload:
        logger.error(
            'You must pass either --forecast_point or '
            '--predictions_start_date and --predictions_end_date'
        )
        return 1

    try:
        make_datarobot_batch_predictions(input_file, output_file, payload,
ssl_insecure, timeout)
    except DataRobotPredictionError as err:
        logger.error('Error: %s', err)
        return 1

    return 0

def make_datarobot_batch_predictions(
    input_file,
    output_file,
    payload,
    ssl_insecure=False,
    timeout=None
):
    # Create new job for batch predictions

```

```

job = _request(
    'POST',
    BATCH_PREDICTIONS_URL,
    data=payload,
    ssl_insecure=ssl_insecure,
    timeout=timeout,
)
links = job['links']

logger.info(
    'Created Batch Prediction job ID {job_id} for deployment ID {deployment_id}'
    ' ({intake} -> {output}) on {self_link}'.format(
        job_id=job['id'],
        deployment_id=DEPLOYMENT_ID,
        intake=job['jobSpec']['intakeSettings']['type'],
        output=job['jobSpec']['outputSettings']['type'],
        self_link=links['self'],
    )
)

# Simultaneously upload
upload_stream = threading.Thread(
    target=upload_datarobot_batch_predictions,
    args=(job, input_file, ssl_insecure, timeout),
)
upload_stream.daemon = True
upload_stream.start()

# Simultaneously download
download_stream = threading.Thread(
    target=download_datarobot_batch_predictions,
    args=(job, output_file, ssl_insecure, timeout),
)
download_stream.daemon = True
download_stream.start()

# Wait until job's complete
job_url = links['self']
while True:
    try:
        job = _request('GET', job_url, ssl_insecure=ssl_insecure,
            timeout=timeout)
        status = job['status']

```

```

if status == JobStatus.INITIALIZING:
    queue_position = job.get("queuePosition")

    if queue_position is None:
        logger.info("Waiting for other jobs to complete")
    elif queue_position > 0:
        logger.info(
            "Waiting for other jobs to complete:
{}".format(queue_position)
        )
    else:
        logger.debug("No queuePosition yet. Waiting for one..")

    _check_logs(job)
    time.sleep(POLL_INTERVAL)
    continue

elif status == JobStatus.RUNNING:
    logger.info(
        "Waiting for the job to complete: {}".format(
            round(float(job["percentageCompleted"]))
        )
    )

    logger.info('Number of scored rows:
{}'.format(job['scoredRows']))
    logger.info('Number of failed rows:
{}'.format(job['failedRows']))
    logger.info('Number of skipped rows:
{}'.format(job['skippedRows']))

    _check_logs(job)
    time.sleep(POLL_INTERVAL)
    continue

elif status in [JobStatus.COMPLETED, JobStatus.ABORTED]:
    upload_stream.join()
    download_stream.join()

    _check_logs(job)
    return

except KeyboardInterrupt:
    print(

```

```

        "KeyboardInterrupt detected, aborting job. "
        "Hang on for a few seconds while we clean up..."
    )
    try:
        _request(
            "DELETE",
            BATCH_PREDICTIONS_URL
            + "{id}/".format(id=job["id"]),
            to_json=False,
            ssl_insecure=ssl_insecure,
            timeout=timeout,
        )
        # It is possible to fail a deletion if the job was COMPLETED before
the client
        # registered it. In that case just ignore it.
    except Exception:
        pass
    return

except Exception as e:
    if 'status' in job and not isinstance(e, DataRobotPredictionError):
        logger.exception('Unexpected error occurred')
        raise DataRobotPredictionError(
            'An unexpected error occurred.\n\n'
            '{err_type}: {err_msg}\n\n'
            'Job {job_id} is {job_status}\n'
            '{job_details}.\nLog: {job_logs}'.format(
                err_type=type(e).__name__,
                err_msg=e,
                job_id=job['id'],
                job_status=job['status'],
                job_details=job['statusDetails'],
                job_logs=job['logs'],
            )
        )
    else:
        raise e


def upload_datarobot_batch_predictions(job_spec, input_file, ssl_insecure,
timeout):
    logger.info('Start uploading csv data')

```



```

upload_url = job_spec['links']['csvUpload']
headers = {
    'Content-length': os.path.getsize(input_file.name),
    'Content-type': 'text/csv; encoding=utf-8',
}
try:
    _request(
        'PUT',
        upload_url,
        data=input_file,
        headers=headers,
        to_json=False,
        ssl_insecure=ssl_insecure,
        timeout=timeout,
    )

except DataRobotPredictionError as err:
    logger.error('Error, attempting to abort the job and exit: %s', err)
    _request(
        "DELETE",
        BATCH_PREDICTIONS_URL
        + "{id}/".format(id=job_spec["id"]),
        to_json=False,
        ssl_insecure=ssl_insecure,
        timeout=timeout,
    )

logger.info('Uploading is finished')

def download_datarobot_batch_predictions(job_spec, output_file, ssl_insecure,
timeout):
    logger.info('Start downloading csv data')

    job_url = job_spec['links']['self']
    job_status = job_spec['status']

    while job_status not in JobStatus.DOWNLOADABLE:
        job_spec = _request('GET', job_url, ssl_insecure=ssl_insecure,
timeout=timeout)
        job_status = job_spec['status']
        time.sleep(1)

    download_url = job_spec['links']['download']

```

```

try:
    with contextlib.closing(
        _request(
            'GET',
            download_url,
            to_json=False,
            ssl_insecure=ssl_insecure,
            timeout=timeout
        )
    ) as response:
        while True:
            chunk = response.read(CHUNK)
            if not chunk:
                break
            output_file.write(chunk)

except DataRobotPredictionError as err:
    logger.error('Error, attempting to abort the job and exit: %s', err)
    _request(
        "DELETE",
        BATCH_PREDICTIONS_URL
        + "{id}/".format(id=job_spec["id"]),
        to_json=False,
        ssl_insecure=ssl_insecure,
        timeout=timeout,
    )

logger.info("Waiting for the job to complete: 100%")
logger.info('Results downloaded to: {}'.format(output_file.name))

def _request(method, url, data=None, headers=None, to_json=True,
             ssl_insecure=False, timeout=None):
    headers = _prepare_headers(headers)

    if isinstance(data, dict):
        data = json.dumps(data).encode('utf-8') # for python3
        headers.update({'Content-Type': 'application/json; encoding=utf-8'})

    ctx = ssl.create_default_context()
    if ssl_insecure:
        ctx.check_hostname = False
        ctx.verify_mode = ssl.CERT_NONE

```

```

request = Request(url, headers=headers, data=data)
request.get_method = lambda: method
try:
    response = urlopen(request, context=ctx, timeout=timeout)
    if to_json:
        result = response.read()
        response.close()

        # json.loads() in 2.7 and prior to 3.6 needed strings, not bytes:
        # https://docs.python.org/3/whatsnew/3.6.html#json.
        if sys.version_info < (3, 6):
            result = result.decode('utf-8')

        return json.loads(result)

    return response

except HTTPError as e:
    err_msg = '{code} Error: {msg}'.format(code=e.code, msg=e.read())
    raise DataRobotPredictionError(err_msg)

except Exception as e:
    err_msg = 'Unhandled exception: {e}'.format(e=e)
    raise DataRobotPredictionError(err_msg)

def _prepare_headers(headers=None):
    if not headers:
        headers = {}
    headers.update({
        'Authorization': 'Bearer {}'.format(API_KEY),
        'User-Agent': 'IntegrationSnippet-Requests',
    })
    return headers

def _check_logs(job):
    global logs_counter
    logs = job['logs']
    if len(logs) > logs_counter:
        new_logs = logs[logs_counter:]
        for log in new_logs:
            logger.info(log)
            logs_counter += 1

```

```

class DataRobotPredictionError(Exception):
    """Raised if there are issues getting predictions from DataRobot"""

class JobStatus(object):
    INITIALIZING = 'INITIALIZING'
    RUNNING = 'RUNNING'
    COMPLETED = 'COMPLETED'
    ABORTED = 'ABORTED'

    DOWNLOADABLE = [RUNNING, COMPLETED, ABORTED]

```

API Client Interface

```

Python
"""
Usage for forecast predictions:
python datarobot-predict.py \
    --forecast_point <date> <input-file.csv> <output-file.csv>

Usage for historical predictions:
python datarobot-predict.py \
    --predictions_start_date <date> --predictions_end_date <date> \
    <input-file.csv> <output-file.csv>

Details:
https://docs.datarobot.com/en/docs/api/reference/batch-prediction-api/index.htm
l
"""

import sys
import logging
import argparse
import datarobot as dr

DEPLOYMENT_ID = '6933846ae8e78154104719ac'

logging.basicConfig(

```

```

        level=logging.INFO,
        stream=sys.stdout,
        format='%(asctime)s %(filename)s:%(lineno)d %(levelname)s %(message)s',
    )
    logger = logging.getLogger(__name__)

def parse_args():
    parser = argparse.ArgumentParser(
        description=__doc__, usage='python %(prog)s <input-file.csv>
<output-file.csv>'
    )
    parser.add_argument('--forecast_point', help='The forecast point, use
"infer" to infer from dataset')
    parser.add_argument('--predictions_start_date', help='Start date for
historical predictions')
    parser.add_argument('--predictions_end_date', help='End date for historical
predictions')
    parser.add_argument(
        'input_file', help='Input CSV file with data to be scored.'
    )
    parser.add_argument(
        'output_file', help='Output CSV file with the scored data.'
    )
    parser.add_argument(
        "--insecure",
        action="store_true",
        default=False,
        dest="ssl_insecure",
        help="Skip SSL certificates verification for HTTPS "
        "endpoints. Using this parameter is not secure and is not recommended.
"
        "This switch is only intended to be used against known hosts using a "
        "self-signed certificate for testing purposes. Use at your own risk.",
    )
    return parser.parse_args()

def main():
    args = parse_args()

    input_file = args.input_file
    output_file = args.output_file
    ssl_insecure = args.ssl_insecure

```

```

timeseries_settings = {}

if args.forecast_point:
    timeseries_settings['type'] = 'forecast'
    if args.forecast_point != 'infer':
        timeseries_settings['forecast_point'] = args.forecast_point

elif args.predictions_start_date and args.predictions_end_date:

    timeseries_settings = {
        'type': 'historical',
        'predictions_start_date': args.predictions_start_date,
        'predictions_end_date': args.predictions_end_date,
    }

if 'type' not in timeseries_settings:
    logger.error(
        'You must pass either --forecast_point or '
        '--predictions_start_date and --predictions_end_date'
    )
    return 1

logger.info(
    "Creating Batch Prediction job for deployment ID
{deployment_id}".format(
        deployment_id=DEPLOYMENT_ID,
    )
)

dr.Client(
    # Uncomment the following lines to run in your own environment
    #     endpoint='https://app.datarobot.com/api/v2',
    #     token='*****',
    ssl_verify=not ssl_insecure,
    user_agent_suffix='IntegrationSnippet-ApiClient'
)

job = dr.BatchPredictionJob.score(
    deployment=DEPLOYMENT_ID,
    intake_settings={
        'type': 'localFile',
        'file': input_file,
    },
    output_settings={

```

```

        'type': 'localFile',
        'path': output_file,
    },
    # If explanations are required, uncomment the line below
    # max_explanations=3,
    # If text explanations are required, uncomment the line below.
    # max_ngram_explanations='all',
    timeseries_settings=timeseries_settings,
    # Uncomment this for Prediction Warnings, if enabled for your
deployment.
    # prediction_warning_enabled=True
)

job.wait_for_completion()

logger.info(
    "Finished Batch Prediction job ID {job_id} for deployment ID
{deployment_id}. "
    "Results downloaded to {output_file}.".format(
        job_id=job.id, deployment_id=DEPLOYMENT_ID, output_file=output_file
    )
)

return 0

if __name__ == '__main__':
    sys.exit(main())

```

Windows Platform

```

Python
.\predict.ps1 -input_file <input_file.csv> -output_file <output_file.csv>
-deployment_id 6933846ae8e78154104719ac -app_host https://app.datarobot.com
-api_key
NjkzM2JlMjdjZDIzOTYwODFjYzgxNGE5OitUZjhHWEdZU1dpMEZqaTBSakFLNWJCTUFDWXNKYTVWTFd
Xem94d1ptSkE9Windows Platform

```